MIPS SIMULATOR in Common Lisp

Reo Tajiri and Daniel Garcia

4/21/2025

Project Overview

- MIPS Simulator written in Common Lisp
- Reads MIPS assembly input file
- Parses and converts instructions to machine code
- Simulates execution of instructions
- Outputs final register and memory state

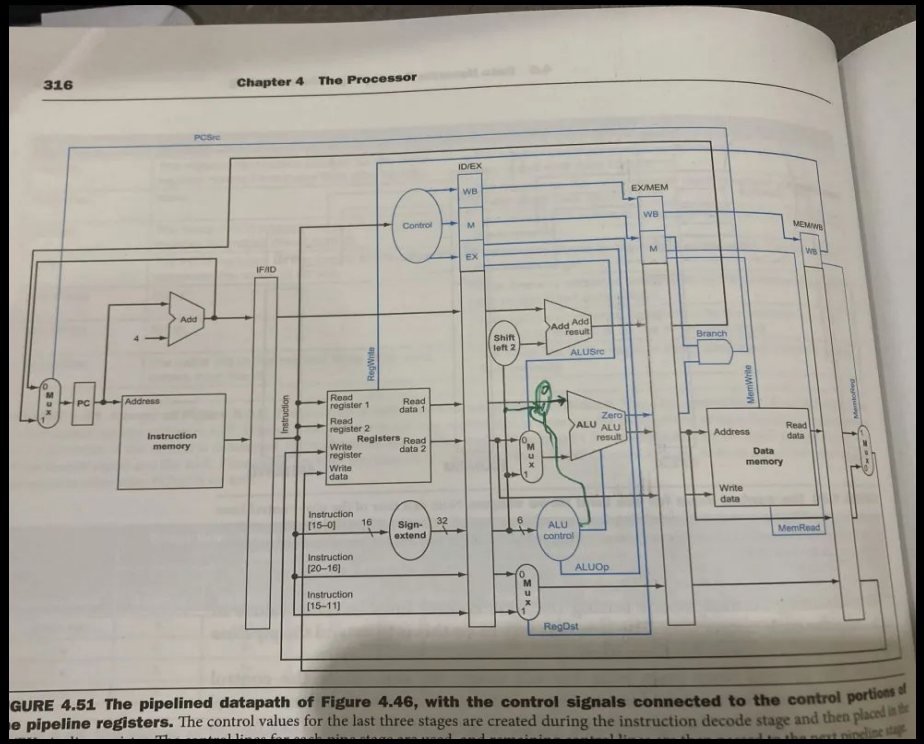
System Flow

1. Parse-assembly
2. Encode
3. Decode
4. Execution
5. Output

Key Functions

- normalize():
  - cleans assembly input
- parse-assembly():
  - splits instruction
- encode():
  - to machine code
- decode():
  - extracts instruction fields
- register/instruction tables
  - map registers/instructions

Design Inspiration



GURE 4.51 The pipelined datapath of Figure 4.46, with the control signals connected to the control portions of e pipeline registers. The control values for the last three stages are created during the instruction decode stage and then placed in the

MIPS Pipeline Datapath Reference

Demo Overview

- Arithmetic Instructions (input-arith)
  - addi, add, sub

- Logic & Shift Instructions (input-logic-shift)
  - and, or, sll, srl

Focused testing of core instruction categories (Executed using SBCL in terminal)

Testing Strategy

- Separate input files for each instruction group:
  - input-arith
  - input-logic-shift

- Unit testing with test.lisp

- Verified correct instruction behavior

Challenges

- Encoding instruction to binary
- Changing from single path execution to pipeline
- Learning how to use LISP and REPL

Conclusion

- Built a MIPS simulator in Lisp
- Demonstrates instruction processing workflow
- Structured testing ensured correctness

QUESTIONS?